

Git & GitHub Basic Assignment

Section A — Guided Assignment

Follow the instructions below step by step.

1. Create an Empty Directory

Create a new empty directory named:

```
Git_Assignment
```

Open a terminal or Git Bash inside the directory.

Verify that the directory is empty.

2. Initialize Git

Initialize a Git repository:

```
git init
```

Check the status:

```
git status
```

3. Create and Commit the First File

Create:

```
notes.txt
```

Add some text of your choice to the file.

Check the repository:

```
git status
```

Stage the file:

```
git add notes.txt
```

Check the status again:

```
git status
```

Commit the file:

```
git commit -m "Add notes file"
```

4. Create and Commit a C Program

Create:

```
program.c
```

Write a simple C program that prints:

```
Hello Git!
```

Check the status, stage the file, and commit it.

```
git add program.c  
git commit -m "Add C program"
```

5. Create and Commit a CSV File

Create:

```
data.csv
```

Add at least **5 rows of sample data**.

Example:

```
ID,Name,Marks  
1,Alice,90  
2,Bob,85  
3,Charlie,95  
4,David,78  
5,Eve,88
```

Stage and commit the file:

```
git add data.csv  
git commit -m "Add sample data"
```

6. Create and Commit a PDF File

Create:

```
report.pdf
```

The PDF can contain any simple content such as:

- Your name
- USN / Roll Number
- Course
- Assignment title

Stage and commit it:

```
git add report.pdf  
git commit -m "Add PDF report"
```

7. Create and Commit an Image

Create or add an image named:

```
image.png
```

The image can be a screenshot, diagram, drawing, or any appropriate image.

Stage and commit it:

```
git add image.png  
git commit -m "Add image"
```

8. Work with .gitignore

Create:

```
.gitignore
```

Add:

```
*.log  
temp/
```

Now create:

```
debug.log
```

Also create a directory:

```
temp
```

Inside `temp`, create:

```
test.txt
```

Run:

```
git status
```

Observe what Git displays.

Answer the following questions:

1. Is `debug.log` displayed in `git status`? Why or why not?
2. Is the `temp` directory displayed in `git status`? Why or why not?
3. What is the purpose of `.gitignore`?
4. Give three examples of files or directories that developers commonly add to `.gitignore`.

Now stage and commit **only** the `.gitignore` file:

```
git add .gitignore  
git commit -m "Add gitignore"
```

9. Examine Your Git History

Run:

```
git log --oneline
```

You should have at least **6 commits**.

Take a screenshot of the output.

10. Create a GitHub Repository

Log in to GitHub and create a new repository named:

```
Git_Assignment
```

Do **not** initialize the GitHub repository with:

- README
- .gitignore
- License

Copy the repository URL.

11. Connect the Local Repository to GitHub

Add your GitHub repository as the remote:

```
git remote add origin https://github.com/<username>/Git_Assignment.git
```

Verify it:

```
git remote -v
```

Set the main branch:

```
git branch -M main
```

Push the repository:

```
git push -u origin main
```

Open GitHub and verify that your files and commit history are visible.

Part 2 — Clone a Repository

12. Clone Your Repository

Create or move to a **different directory outside your current `Git_Assignment` directory**.

Clone the repository you just uploaded:

```
git clone https://github.com/<username>/Git_Assignment.git
```

Enter the cloned directory:

```
cd Git_Assignment
```

Check:

```
git status
```

View the history:

```
git log --oneline
```

Answer:

1. What does `git clone` do?
 2. Did the cloned repository contain your previous commit history?
 3. What is the difference between downloading a ZIP file from GitHub and using `git clone`?
-

13. Make a Change in the Cloned Repository

Modify:

```
notes.txt
```

Add one new line of text.

Run:

```
git status
```

Commit the change:

```
git add notes.txt  
git commit -m "Update notes after cloning"
```

Push it:

```
git push
```

Verify the new commit on GitHub.

Part 3 — Fork a Repository

14. Fork Another Repository

Find the GitHub repository of another student participating in the assignment.

Open their repository on GitHub and click **Fork**.

Create the fork under your own GitHub account.

You should now have your own copy of their repository under your GitHub account.

15. Clone Your Fork

Copy the URL of **your fork**, not the original student's repository.

Clone it:

```
git clone https://github.com/<your-username>/<forked-repository>.git
```

Enter the repository:

```
cd <forked-repository>
```

Check:

```
git remote -v
```

Answer:

1. What is a fork?
 2. Who owns the original repository?
 3. Who owns the forked repository?
 4. What is the difference between **forking** and **cloning**?
-

16. Modify the Forked Repository

Create a new text file:

```
contribution.txt
```

Add:

```
This contribution was made by <Your Name>.
```

Stage and commit it:

```
git add contribution.txt
git commit -m "Add contribution"
```

Push the change:

```
git push
```

Verify that `contribution.txt` appears in **your fork** on GitHub.

Section B — Try It Yourself

For this section, **no commands or step-by-step instructions are provided.**

Use what you learned in Section A to complete the following tasks independently.

Challenge

1. Create a New Repository

Create a new empty directory named:

```
Git_Challenge
```

Initialize it as a Git repository.

2. Create Files

Create at least **5 files of different types.**

You decide:

- The filenames
- The file extensions
- The contents of each file

3. Build a Commit History

Add the files to Git **one at a time.**

Each file must have its own commit.

Use meaningful commit messages.

Your repository must contain at least **5 commits.**

4. Use `.gitignore`

Create your own `.gitignore`.

Configure it to ignore:

- At least one specific file
- At least one file extension
- At least one directory

Create those files/directories and prove using `git status` that Git ignores them.

5. GitHub

Create a new GitHub repository for the challenge.

Connect your local repository to it and push your complete commit history.

6. Clone

Clone your GitHub repository into a different location on your computer.

Make a change inside the cloned repository, commit it, and push the change back to GitHub.

7. Fork

Fork another student's **Git_Challenge** repository.

Clone your fork to your computer.

Make one meaningful change to the project.

Commit the change and push it to your fork.

8. Pull Request

After pushing your change to the fork, create a **Pull Request** on GitHub requesting that the original repository owner accept your change.

Do this without step-by-step instructions.

Submission

Submit **two GitHub repository links**:

1. Your `Git_Assignment` repository from Section A.
2. Your `Git_Challenge` repository from Section B.

Also submit screenshots showing:

- `git log --oneline` for Section A.
- `git log --oneline` for Section B.
- Your `.gitignore` test.
- Successful push to GitHub.
- Your cloned repository.
- Your fork on GitHub.
- The Pull Request created in Section B.

Short Questions

Answer briefly in your own words:

1. What is Git?
2. What is GitHub?
3. What is a Git repository?
4. What is the staging area?
5. What is a commit?
6. What does `.gitignore` do?
7. What does `git clone` do?
8. What is a fork?
9. What is the difference between a fork and a clone?
10. What is the difference between `git add`, `git commit`, and `git push`?
11. What is a remote repository?
12. What is a Pull Request?